Coding Sample: Escaping Inequality in India

Selected Excerpts

Rishav Roy

What follows are excerpts from the 138-page code appendix to my independent paper and writing sample, "Escaping Inequality in India: The Role of English-Medium Instruction."

The next 25 pages illustrate data ingestion and harmonization, complex survey econometrics, IV estimation, and careful diagnostics. Each excerpt is a part of the code chunks listed below:

14-16. Scalable **import, cleaning, and merging** of multi-file census and geospatial data.

- Chunk 4: Import key data files

31-32. **QA and identifier-disambiguation checks** for district-level crosswalk.

- Chunk 6: Match districts: Construct district tracking dfs

50-52. **Testing for systematic missingness** using the Benjamini–Hochberg procedure and parallelized logistic regressions.

- Chunk 8: Participation model: Are NAs randomly distributed

55-57. **Survey-weighted probit estimation**, average marginal effects derivation, and setup for sample selection correction under a complex survey design.

- Chunk 9: Participation model: Estimation and IMR
- Chunk 10: Participation model: Derive AME

80-88. Reusable **record-linkage functions** using cascading fuzzy matches and unmatched/false-match diagnostics to harmonize district identifiers across years and data sources.

- Chunk 16: Match districts: Test joining methods
- Chunk 17: Match districts: Define joining functions

105-107. Reusable **IV specification and estimation pipeline** with clustered inference, plus contiguity-based spatial weights and setup for spatially lagged models.

- Chunk 24: Geospatial: Neighbor list construction
- Chunk 25: 2SLS: Controls and 2SLS formula
- Chunk 26: 2SLS: Baseline 2SLS models

111-112. **Robustness checks and diagnostics** for multicollinearity, spatial autocorrelation.

- Chunk 28: Multicollinearity check
- Chunk 29: Geospatial: Spatial autocorrelation tests

Scalable import, cleaning, and merging of Census/geospatial data

Source: `R/io/read_inputs.R`

```
#' read nss 2007 education
#'
read_nss_2007_education <- function(paths) {
  read_manifest_group(paths, "nss_2007_education")
}

#' read nss 2007 consumption
#'
read_nss_2007_consumption <- function(paths) {
  read_manifest_group(paths, "nss_2007_consumption")
}

#' read nss 2017 education
#'
read_nss_2017_education <- function(paths) {
  read_manifest_group(paths, "nss_2017_education")
}

#' read census 2001 mother tongue
#'
read_census_2001_mother_tongue <- function(paths) {
  read_manifest_group(paths, "census_2001_mother_tongue")
}

#' read district boundaries 2020
#'
read_district_boundaries_2020 <- function(paths) {
  rows <- require_manifest_files(paths, "district_boundaries_2020")
  shp <- rows[tolower(rows$file_type) == "shp", , drop = FALSE]
  if (!nrow(shp)) stop("The district-boundary manifest includes shapefile sidecars but no
   ↪  .shp row.", call. = FALSE)
  need_pkg("sf", "district boundaries")
  sf::st_read(shp$absolute_path[[1]], quiet = TRUE)
}

#' read district change sources
#'
read_district_change_sources <- function(paths) {
  read_manifest_group(paths, "district_changes")
}

#' list ilo figure paths
#'
list_ilo_figure_paths <- function(paths) {
  rows <- require_manifest_files(paths, "ilo_figures")
  stats::setNames(rows$absolute_path, rows$file_id)
}

#' read one manifest row
#'
#' @return Reader output for raw data files, or a validated path for sidecars/assets.
read_by_manifest_row <- function(row) {
```

```
  path <- row$absolute_path[[1]]
  file_type <- tolower(row$file_type[[1]])
  switch(
    file_type,
    sav = read_sav_short(path),
    csv = read_csv_short(path),
    xls = read_excel_short(path),
    xlsx = read_excel_short(path),
    ods = read_ods_short(path),
    shp = {
      need_pkg("sf", "shapefiles")
      sf::st_read(path, quiet = TRUE)
    },
    shx = path,
    dbf = path,
    prj = path,
    png = path,
    stop("No reader implemented for ", file_type, call. = FALSE)
  )
}

#' read all manifest rows for a source
#'
#' @return Named list of reader outputs keyed by manifest file_id.
read_manifest_group <- function(paths, source_id) {
  rows <- require_manifest_files(paths, source_id)
  stats::setNames(lapply(seq_len(nrow(rows)), function(i) read_by_manifest_row(rows[i,
  ↪ ])), rows$file_id)
}
```

QA and identifier-disambiguation checks

Source: `R/districts/build_district_tracker.R`

```
#' build district tracker
#'
build_district_tracker <- function(raw_district_changes) {
  combine_district_tracker_sources(raw_district_changes)
}

#' parse alluvial district changes
#'
parse_alluvial_district_changes <- function(x) {
  x
}

#' parse india district tracker
#'
parse_india_district_tracker <- function(x) {
  x
}

#' parse carveouts renamings
#'
parse_carveouts_renamings <- function(x) {
  x
```

```
}

#' parse new districts created
#'
parse_new_districts_created <- function(x) {
  x
}

#' parse name changes
#'
parse_name_changes <- function(x) {
  x
}

#' parse district splits
#'
parse_district_splits <- function(x) {
  x
}

#' combine district tracker sources
#'
combine_district_tracker_sources <- function(raw_district_changes) {
  out <- safe_bind_rows(lapply(names(raw_district_changes), function(name) {
    x <- safe_df(raw_district_changes[[name]])
    x$source_file_id <- name
    x
  }))
  if (nrow(out)) out$.row_in_source <- ave(seq_len(nrow(out)), out$source_file_id, FUN =
   ↪  seq_along)
  out
}

#' standardize tracker years
#'
standardize_tracker_years <- function(tracker) {
  tracker
}

#' standardize tracker names
#'
standardize_tracker_names <- function(tracker) {
  tracker
}
```

Parallelized missingness diagnostics with BH correction

Source: `R/selection/diagnose_missingness.R`

```
#' diagnose missingness
#'
diagnose_missingness <- function(selection_data, cfg) {
  data.frame(
    missing_var = names(selection_data)[vapply(selection_data, function(x) any(is.na(x)),
     ↪  logical(1))],
    stringsAsFactors = FALSE
```

```
  )
}

#' check missing logit parallel
#'
check_missing_logit_parallel <- function(df, miss_vars, covars, method_p = "BH") {
  rhs <- paste(covars, collapse = " + ")
  fit_one <- function(m) { f <- stats::as.formula(paste0("is.na(", m, ") ~ ", rhs)); fit
 ↪  <- stats::glm(f, data = df, family = stats::binomial); broom::tidy(fit) |>
 ↪  dplyr::mutate(missing_var = m, pseudoR2 = 1 - fit$deviance / fit$null.deviance, nobs
 ↪  = stats::nobs(fit)) }
  out <- dplyr::bind_rows(lapply(miss_vars, fit_one))
  out |> dplyr::group_by(missing_var) |> dplyr::mutate(p_adj = {adj <- rep(NA_real_,
 ↪  dplyr::n()); idx <- term != "(Intercept)"; adj[idx] <- p.adjust(p.value[idx], method
 ↪  = method_p); adj}) |> dplyr::ungroup()
}

#' summarize missingness by variable
#'
summarize_missingness_by_variable <- function(df) {
  data.frame(
    missing_var = names(df),
    n_missing = vapply(df, function(x) sum(is.na(x)), integer(1)),
    stringsAsFactors = FALSE
  )
}

#' run missingness logits
#'
run_missingness_logits <- function(df, miss_vars, covars) {
  check_missing_logit_parallel(df, miss_vars, covars)
}

#' adjust missingness pvalues bh
#'
adjust_missingness_pvalues_bh <- function(df) {
  df
}

#' save missingness diagnostics
#'
save_missingness_diagnostics <- function(diagnostics) {
  diagnostics
}
```

Survey-weighted probit and IMR setup

Source: `R/selection/estimate_selection_probit.R`

```
#' estimate selection probit
#'
estimate_selection_probit <- function(selection_data, cfg) {
  if (!"enrolled" %in% names(selection_data) || all(is.na(selection_data$enrolled))) {
    return(list(status = "out_of_active_pipeline", reason = "No enrolled variable."))
  }
  covars <- intersect(
```

```
    c(
      "AGE", "SEX", "HH_SIZE", "RELIGION", "SOCIAL_GROUP", "SECTOR",
      "age", "sex", "hh_size", "religion", "social_group", "sector",
      "DIST_FROM_NEAREST_PRIMARY_CLASS",
      "dmean_num_IS_EDU_FREE", "dmean_num_RECD_TXT_BOOKS"
    ),
    names(selection_data)
  )
  if (!length(covars)) {
    return(list(status = "out_of_active_pipeline", reason = "No probit covariates."))
  }
  f_probit <- stats::as.formula(paste("enrolled ~", paste(covars, collapse = "+")))
  if (identical(cfg$mode, "final") && requireNamespace("survey", quietly = TRUE)) {
    design <- build_survey_design_selection(selection_data)
    if (!is.null(design)) {
      return(fit_selection_probit(design, f_probit))
    }
  }
  stats::glm(f_probit, data = selection_data, family = stats::binomial(link = "probit"))
}

#' build survey design selection
#'
build_survey_design_selection <- function(selection_df) {
  psu <- first_col(selection_df, c("FSU_SL_NO", "fsu", "PSU", "psu"))
  weight <- first_col(selection_df, c("weight", "WEIGHT", "Multiplier", "multiplier"))
  strata_cols <- intersect(c("STATE", "state_std", "STRATUM", "SUB_STRATUM_NO"),
 ↪  names(selection_df))
  if (is.null(psu) || is.null(weight) || !length(strata_cols)) return(NULL)
  options(survey.lonely.psu = "average")
  strata <- interaction(selection_df[strata_cols], drop = TRUE)
  selection_df$.survey_strata <- strata
  survey::svydesign(
    ids = stats::as.formula(paste0("~", psu)),
    strata = ~.survey_strata,
    weights = stats::as.formula(paste0("~", weight)),
    data = selection_df,
    nest = TRUE
  )
}

#' fit selection probit
#'
fit_selection_probit <- function(selection_design, f_probit) {
  survey::svyglm(f_probit, design = selection_design, family = quasibinomial(link =
 ↪  "probit"))
}

#' compute inverse mills ratio
#'
compute_inverse_mills_ratio <- function(model, selection_df, f_probit) {
  selection_df
}

#' tidy selection model
```

```r
#'
tidy_selection_model <- function(model) {
  broom::tidy(model)
}
```

Average marginal effects via automatic differentiation

Source: `R/selection/compute_average_marginal_effects.R`

```r
# Use automatic differentiation for SE delta-method gradients, as in the legacy Rmd.

#' compute average marginal effects
#'
compute_average_marginal_effects <- function(selection_model, selection_data, cfg =
 ↪  list()) {
  if (!inherits(selection_model, "glm")) {
    return(ame_out_of_pipeline(
      selection_model$status %||% "out_of_active_pipeline",
      selection_model$reason %||% "Selection model not estimated in smoke mode"
    ))
  }

  if (!isTRUE(cfg$run_full_ame)) {
    return(format_ame_results(data.frame(
      term = names(stats::coef(selection_model)),
      estimate = unname(stats::coef(selection_model)),
      std.error = NA_real_,
      statistic = NA_real_,
      p.value = NA_real_,
      s.value = NA_real_,
      conf.low = NA_real_,
      conf.high = NA_real_,
      method = "coefficient_fallback",
      status = "estimated",
      reason = "Draft config uses coefficient fallback instead of full AME computation"
    )))
  }

  if (!requireNamespace("marginaleffects", quietly = TRUE)) {
    return(ame_out_of_pipeline(
      "out_of_active_pipeline",
      "Package marginaleffects is required for full AME computation"
    ))
  }

  out <- tryCatch(
    compute_ames_autodiff(selection_model, selection_data),
    error = function(e) {
      fallback <- compute_ames_probit_analytic(selection_model, selection_data)
      fallback$method <- "delta_method_analytic_probit"
      fallback$reason <- NA_character_
      fallback
    }
  )
  format_ame_results(out)
}
```

```r
ame_out_of_pipeline <- function(status, reason) {
  data.frame(
    term = NA_character_,
    estimate = NA_real_,
    std.error = NA_real_,
    statistic = NA_real_,
    p.value = NA_real_,
    s.value = NA_real_,
    conf.low = NA_real_,
    conf.high = NA_real_,
    method = "not_run",
    status = status,
    reason = reason
  )
}

#' extract model-frame data and AME weights
#'
#' marginaleffects warns, correctly, when weighted/survey models are summarized
#' without an explicit `wts` argument. Build a newdata frame from the exact
#' model estimation sample and attach a synthetic weight column whenever the
#' fitted model exposes usable weights.
ame_model_weight <- function(model_data, model_weights = NULL) {
  weight_cols <- intersect(c("weight", "WEIGHT", "Multiplier", "multiplier",
 ↪  "(weights)"), names(model_data))
  if (length(weight_cols)) {
    return(suppressWarnings(as.numeric(model_data[[weight_cols[[1]]]])))
  }

  if (!is.null(model_weights) && length(model_weights) == nrow(model_data)) {
    return(suppressWarnings(as.numeric(model_weights)))
  }

  NULL
}

#' build marginaleffects newdata and weights
#'
#' @return List with `data`, `wts`, and `has_explicit_weights`.
ame_model_data_and_weights <- function(model) {
  model_data <- as.data.frame(stats::model.frame(model))
  model_weights <- tryCatch(stats::weights(model), error = function(e) NULL)
  weight <- ame_model_weight(model_data, model_weights)

  if (!is.null(weight) && length(weight) == nrow(model_data) && any(is.finite(weight) &
   ↪  weight != 1)) {
    model_data$.ame_weight <- weight
    return(list(data = model_data, wts = ".ame_weight", has_explicit_weights = TRUE))
  }

  list(data = model_data, wts = FALSE, has_explicit_weights = FALSE)
}

#' run marginaleffects while muffling only a redundant explicit-weight warning
```

```r
#'
run_avg_slopes <- function(model, model_data, wts) {
  withCallingHandlers(
    marginaleffects::avg_slopes(
      model,
      newdata = model_data,
      wts = wts,
      vcov = TRUE,
      type = "response"
    ),
    warning = function(w) {
      msg <- conditionMessage(w)
      explicit_weight_warning <- grepl(
        "normally good practice to specify weights using the `wts` argument",
        msg,
        fixed = TRUE
      )
      if (explicit_weight_warning && !identical(wts, FALSE)) {
        invokeRestart("muffleWarning")
      }
    }
  )
}

#' compute ames autodiff
#'
compute_ames_autodiff <- function(model, newdata) {
  # Use the exact estimation sample retained by glm/svyglm. Passing the full
  # pre-model selection_data can have extra rows omitted during model fitting,
  # which makes marginaleffects recycle weights/covariates silently.
  amed <- ame_model_data_and_weights(model)
  run_avg_slopes(model, amed$data, amed$wts)
}

#' compute ames fast draft
#'
compute_ames_fast_draft <- function(model, newdata, n = 200) {
  amed <- ame_model_data_and_weights(model)
  run_avg_slopes(
    model,
    dplyr::slice_sample(amed$data, n = min(n, nrow(amed$data))),
    amed$wts
  )
}

#' compute analytic probit AMEs
#'
#' @return Data frame of observed-data average marginal effects.
compute_ames_probit_analytic <- function(model, newdata) {
  coefs <- stats::coef(model)
  coef_table <- as.data.frame(summary(model)$coefficients)
  terms <- setdiff(names(coefs), "(Intercept)")
  if (!length(terms)) return(ame_out_of_pipeline("out_of_active_pipeline", "Selection
   ↪  model has no non-intercept terms."))
```

```
  newdata <- stats::model.frame(model)
  weights <- if ("weight" %in% names(newdata)) num(newdata$weight) else rep(1,
↪  nrow(newdata))
  eta <- stats::predict(model, type = "link")
  mean_phi <- wmean(stats::dnorm(eta), weights)
  factor_levels <- model$xlevels %||% list()

  rows <- lapply(terms, function(term) {
    factor_var <- names(factor_levels)[vapply(names(factor_levels), function(v)
↪  startsWith(term, v), logical(1))]
    estimate <- NA_real_
    if (length(factor_var)) {
      var <- factor_var[[which.max(nchar(factor_var))]]
      level <- sub(paste0("^", var), "", term)
      if (level %in% factor_levels[[var]]) {
        hi <- newdata
        lo <- newdata
        hi[[var]] <- factor(level, levels = factor_levels[[var]])
        lo[[var]] <- factor(factor_levels[[var]][[1]], levels = factor_levels[[var]])
        estimate <- wmean(stats::predict(model, newdata = hi, type = "response") -
↪  stats::predict(model, newdata = lo, type = "response"), weights)
      }
    } else {
      estimate <- unname(coefs[[term]]) * mean_phi
    }
    p_col <- intersect(c("Pr(>|z|)", "Pr(>|t|)"), names(coef_table))
    p <- if (term %in% rownames(coef_table) && length(p_col)) coef_table[term,
↪  p_col[[1]]] else NA_real_
    se_col <- intersect(c("Std. Error", "std.error"), names(coef_table))
    se_beta <- if (term %in% rownames(coef_table) && length(se_col)) coef_table[term,
↪  se_col[[1]]] else NA_real_
    se <- if (is.finite(se_beta)) abs(se_beta * mean_phi) else NA_real_
    z <- if (is.finite(se) && se > 0) estimate / se else NA_real_
    p_out <- if (is.finite(z)) 2 * stats::pnorm(abs(z), lower.tail = FALSE) else p
    data.frame(
      term = term,
      estimate = estimate,
      std.error = se,
      statistic = z,
      p.value = p_out,
      s.value = if (is.finite(p_out) && p_out > 0) -log2(p_out) else NA_real_,
      conf.low = if (is.finite(se)) estimate - 1.96 * se else NA_real_,
      conf.high = if (is.finite(se)) estimate + 1.96 * se else NA_real_,
      method = "delta_method_analytic_probit",
      status = "estimated",
      reason = NA_character_,
      stringsAsFactors = FALSE
    )
  })
  do.call(rbind, rows)
}

copy_first_ame_column <- function(out, target, candidates) {
  if (!target %in% names(out)) out[[target]] <- NA
  for (candidate in candidates) {
```

```
    if (!candidate %in% names(out)) next
    missing_target <- is.na(out[[target]])
    if (any(missing_target)) out[[target]][missing_target] <-
     ↪  out[[candidate]][missing_target]
  }
  out
}

normalize_ame_columns <- function(out) {
  # marginaleffects has used both dotted and snake_case names across versions
  # and model classes. Normalize here before selecting the public/audit schema
  # so uncertainty columns are not silently dropped downstream.
  aliases <- list(
    std.error = c("std_error", "std.err", "Std. Error"),
    statistic = c("z", "z.value", "z_value", "t", "t.value", "t_value"),
    p.value = c("p_value", "p", "Pr(>|z|)", "Pr(>|t|)"),
    s.value = c("s_value"),
    conf.low = c("conf_low", "conf.low", "2.5 %"),
    conf.high = c("conf_high", "conf.high", "97.5 %")
  )
  for (target in names(aliases)) {
    out <- copy_first_ame_column(out, target, aliases[[target]])
  }
  out
}

#' format ame results
#'
format_ame_results <- function(ame_results) {
  out <- tibble::as_tibble(ame_results)
  if (!"term" %in% names(out) && "variable" %in% names(out)) out$term <- out$variable
  out <- normalize_ame_columns(out)
  if (!"method" %in% names(out)) out$method <- "autodiff"
  if (!"status" %in% names(out)) out$status <- "estimated"
  if (!"reason" %in% names(out)) out$reason <- NA_character_

  required <- c(
    "term", "estimate", "std.error", "statistic", "p.value", "s.value",
    "conf.low", "conf.high", "method", "status", "reason"
  )
  for (nm in setdiff(required, names(out))) out[[nm]] <- NA
  if ("p.value" %in% names(out) && "s.value" %in% names(out)) {
    missing_s <- is.na(out$s.value) & is.finite(out$p.value) & out$p.value > 0
    out$s.value[missing_s] <- -log2(out$p.value[missing_s])
  }
  out[, required, drop = FALSE]
}

#' save ame results
#'
save_ame_results <- function(ame_results, path =
 ↪  "outputs/tables/diagnostics/ame_results.csv") {
  readr::write_csv(tibble::as_tibble(ame_results), path); path
}
```

Cascading fuzzy district record linkage

Source: `R/districts/fuzzy_join_districts.R`

```
# The functions below preserve the cascading fuzzy-match architecture from the legacy
↪  Rmd.
# sample-end marker appears near the end of this file after the functions are migrated.

#' evaluate distances
#'
evaluate_distances <- function(pairs, methods, thresholds, col1 = "str1", col2 = "str2")
↪  {
  if (length(methods) != length(thresholds)) stop("\"methods\" and \"thresholds\" must
  ↪  have the same length.")
  safe_bind_rows(lapply(seq_along(methods), function(i) {
    distance <- utils::adist(canon(pairs[[col1]]), canon(pairs[[col2]]), ignore.case =
↪  TRUE)[, 1]
    data.frame(
      str1 = pairs[[col1]],
      str2 = pairs[[col2]],
      method = methods[i],
      distance = distance,
      threshold = thresholds[i],
      match = distance <= thresholds[i],
      stringsAsFactors = FALSE
    )
  }))
}

#' fuzzy join sequence
#'
fuzzy_join_sequence <- function(df1, df2, dist1, state1, dist2, state2, methods,
↪  thresholds, mode = "full") {
  df1_id <- df1 |> dplyr::mutate(.id1 = dplyr::row_number())
  df2_id <- df2 |> dplyr::mutate(.id2 = dplyr::row_number())
  df1_curr <- df1_id; df2_curr <- df2_id; matched_all <- tibble::tibble()
  for (i in seq_along(methods)) {
    full_j <- fuzzyjoin::stringdist_join(df1_curr, df2_curr, by =
↪  stats::setNames(c(dist2, state2), c(dist1, state1)), mode = mode, method =
↪  methods[i], max_dist = thresholds[i], distance_col = "dist")
    matched_i <- full_j |> dplyr::filter(!is.na(.id1), !is.na(.id2)) |>
↪  dplyr::group_by(.id1) |> dplyr::slice_min(dist, n = 1, with_ties = FALSE) |>
↪  dplyr::ungroup() |> dplyr::group_by(.id2) |> dplyr::slice_min(dist, n = 1, with_ties
↪  = FALSE) |> dplyr::ungroup()
    matched_all <- dplyr::bind_rows(matched_all, matched_i)
    df1_curr <- df1_curr |> dplyr::filter(!(.id1 %in% matched_i$.id1)); df2_curr <-
↪  df2_curr |> dplyr::filter(!(.id2 %in% matched_i$.id2))
  }
  list(joined = matched_all, unmatched_df1 = df1_curr |> dplyr::select(-.id1),
  ↪  unmatched_df2 = df2_curr |> dplyr::select(-.id2))
}

#' unmatched rows
#'
#' @return A data frame of unmatched rows when the join map carries that attribute.
unmatched_rows <- function(join_map, ...) {
```

```r
  attr(join_map, "unmatched_rows") %||% join_map[0, , drop = FALSE]
}

#' merge dfs into tracker
#'
#' @return A row-bound tracker data frame from source-specific tracker fragments.
merge_dfs_into_tracker <- function(...) {
  safe_bind_rows(list(...))
}

#' join year to tracker
#'
#' @return Explicit inactive status for the future source-specific tracker join.
join_year_to_tracker <- function(...) {
  data.frame(
    status = "out_of_active_pipeline",
    reason = "Source-specific year-to-tracker joins are not active;
     ↪  fuzzy_join_districts() consumes normalized keys directly.",
    stringsAsFactors = FALSE
  )
}

#' join all district sources
#'
#' @return Explicit inactive status for the future all-source tracker join.
join_all_district_sources <- function(...) {
  data.frame(
    status = "out_of_active_pipeline",
    reason = "All-source district joining is represented by fuzzy_join_districts() in the
     ↪  active pipeline.",
    stringsAsFactors = FALSE
  )
}

#' flag many to many matches
#'
flag_many_to_many_matches <- function(join_map) {
  join_map
}

#' score candidate matches
#'
score_candidate_matches <- function(candidates) {
  candidates
}

#' fuzzy join districts
#'
fuzzy_join_districts <- function(district_tracker, district_keys_2001,
 ↪  district_keys_2007, district_keys_2017, district_keys_2020, cfg) {
  out <- safe_bind_rows(Map(function(keys, source) {
    if (!nrow(keys)) return(data.frame())
    keys$source <- source
    keys$match_status <- "key_only"
    keys$possible_false_positive <- FALSE
```

```
    keys$many_to_many <- FALSE
    keys
  }, list(district_keys_2001, district_keys_2007, district_keys_2017,
 ↪  district_keys_2020), c("2001", "2007", "2017", "2020")))
  attr(out, "unmatched_rows") <- out[0, , drop = FALSE]
  attr(out, "possible_false_positives") <- out[out$possible_false_positive, , drop =
   ↪  FALSE]
  attr(out, "many_to_many_cases") <- out[out$many_to_many, , drop = FALSE]
  out
}
```

Contiguity-based spatial weights

Source: R/diagnostics/diagnose_spatial_weights.R

```
#' build spatial weights
#'
#' @return A spatial weights object, or an explicit inactive status when geometry is
 ↪  unavailable.
build_spatial_weights <- function(district_panel, cfg) {
  if (!inherits(district_panel, "sf")) {
    return(list(status = "out_of_active_pipeline", reason = "Requires sf geometry."))
  }
  geom <- sf::st_geometry(district_panel)
  coverage <- mean(!sf::st_is_empty(geom))
  if (!is.finite(coverage) || coverage < 0.75) {
    return(list(
      status = "out_of_active_pipeline",
      reason = paste0(
        "Requires validated non-empty geometry for at least 75% of district-panel rows;
         ↪  current coverage is ",
        round(100 * coverage, 1), "%.")
    ))
  }
  nb <- spdep::poly2nb(district_panel[!sf::st_is_empty(geom), , drop = FALSE], queen =
 ↪  FALSE)
  spdep::nb2listw(nb, style = "W", zero.policy = TRUE)
}

#' diagnose spatial weights
#'
diagnose_spatial_weights <- function(district_panel, spatial_weights, cfg) {
  data.frame(
    diagnostic = "spatial_weights",
    status = spatial_weights$status %||% "constructed",
    stringsAsFactors = FALSE
  )
}

#' compare rook queen contiguity
#'
compare_rook_queen_contiguity <- function(district_panel) {
  tibble::tibble()
}

#' summarize islands
```

```r
#'
summarize_islands <- function(spatial_weights) {
  tibble::tibble()
}

#' summarize neighbor counts
#'
summarize_neighbor_counts <- function(spatial_weights) {
  tibble::tibble()
}

#' save spatial weight diagnostics
#'
save_spatial_weight_diagnostics <- function(diagnostics) {
  diagnostics
}
```

Reusable IV specification and estimation

Source: `R/iv/build_iv_formulas.R`

```r
#' make iv formula
#'
make_iv_formula <- function(dep, endog, instruments, controls = NULL, fixed_effects =
 ↪  NULL) {
  stats::as.formula(paste(
    dep,
    "~",
    paste(c(endog, controls, fixed_effects), collapse = " + "),
    "|",
    paste(c(instruments, controls, fixed_effects), collapse = " + ")
  ))
}

#' build iv formulas
#'
build_iv_formulas <- function(cfg) {
  controls <- c(
    "consumption_2007", "gini_consumption_2007",
    "pct_urban", "avg_hh_size", "dependency_ratio",
    "pct_fem_head",
    "pct_hindu", "pct_muslim",
    "pct_st", "pct_sc", "pct_obc",
    "pct_small_land", "pct_medium_land", "pct_large_land",
    "pct_head_lit_to_primary",
    "pct_head_secondary_plus"
  )
  list(
    baseline = make_iv_formula(
      "consumption_growth_pct",
      "emie_2007",
      "wavg_ling_degrees",
      controls
    ),
    fd_log = make_iv_formula(
      "log_consumption_difference",
```

```
      "emie_2007",
      "wavg_ling_degrees",
      controls
    )
  )
}

#' build baseline 2sls formula
#'
build_baseline_2sls_formula <- function(...) {
  make_iv_formula(...)
}

#' build fd 2sls formula
#'
#' @return Explicit inactive status until the first-difference redesign is specified.
build_fd_2sls_formula <- function(...) {
  list(status = "out_of_active_pipeline", reason = "First-difference 2SLS formula is
   ↪  deferred until the FD redesign is specified.")
}

#' build state fe 2sls formula
#'
#' @return Explicit inactive status until the state fixed-effect redesign is specified.
build_state_fe_2sls_formula <- function(...) {
  list(status = "out_of_active_pipeline", reason = "State fixed-effect 2SLS formula is
   ↪  deferred until the state-FE redesign is specified.")
}
```

Multicollinearity and spatial autocorrelation diagnostics

Source: `R/diagnostics/diagnose_multicollinearity.R`

```
#' diagnose multicollinearity
#'
#' @return A data frame with design-matrix condition diagnostics.
diagnose_multicollinearity <- function(district_panel, iv_models, cfg) {
  model <- if (is.list(iv_models) && length(iv_models)) iv_models[[1]] else iv_models
  out <- tryCatch({
    X <- stats::model.matrix(model)
    data.frame(
      test = "kappa",
      n = nrow(X),
      rank = qr(X)$rank,
      columns = ncol(X),
      kappa = kappa(X, exact = TRUE),
      status = "estimated",
      reason = NA_character_,
      stringsAsFactors = FALSE
    )
  }, error = function(e) {
    data.frame(
      test = "kappa",
      n = NA_integer_,
      rank = NA_integer_,
      columns = NA_integer_,
```

```
      kappa = NA_real_,
      status = "out_of_active_pipeline",
      reason = conditionMessage(e),
      stringsAsFactors = FALSE
    )
  })
  out
}

#' compute design matrix rank
#'
compute_design_matrix_rank <- function(formula, data) {
  X <- stats::model.matrix(formula, data = data); tibble::tibble(rank = qr(X)$rank, ncol
 ↪  = ncol(X))
}

#' compute kappa
#'
compute_kappa <- function(formula, data) {
  X <- stats::model.matrix(formula, data = data); kappa(X, exact = TRUE)
}

#' compute vif if applicable
#'
compute_vif_if_applicable <- function(model) {
  if (requireNamespace("car", quietly = TRUE)) car::vif(model) else NULL
}

#' save multicollinearity diagnostics
#'
save_multicollinearity_diagnostics <- function(diagnostics) {
  diagnostics
}
```